

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	DINESH KARTHICK R	Reg. No.:	192524121
Section / Batch:	02	Date:	24/07/2026

Instructions to Students

- Answer BOTH scenario-based questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions	Marks
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q31 & Q37	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – SCENARIO BASED ASSESSMENT

Q31. A data analytics platform processes sensor readings that arrive in random order and must be sorted periodically for trend visualization.

- Analyze how Selection Sort and Bubble Sort would behave for this dataset.
- Compare their performance in terms of comparisons and swaps.
- Recommend the most suitable sorting method with proper justification.

Q37. A satellite tracking system evaluates coordinates of objects in space to find the closest pair using brute-force comparison.

- Explain how all coordinate pairs are evaluated.
- Derive the time complexity of the brute-force method.
- Discuss the limitations when the number of objects increases.

Code of Conduct

I DINESH KARTHICK R / 192524121 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Question No.	Problem Understanding & Context Analysis (25%)	Application of Concepts (30%)	Analytical & Decision-Making Skills (20%)	Solution Design & Approach (15%)	Clarity, Justification & Presentation (10%)	Total Marks (Each – 50)
Q31	4	3	3	2	4	40
Q37	4	4	4	3	2	46
Overall Total (100)						86

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

ANSWER SHEET — SECTION A

Student Name: DINESH KARTHICK R Register Number: 192524121 Section / Batch: 02 Date: 24/07/2026

Q31. A data analytics platform processes sensor readings that arrive in random order and must be sorted periodically for trend visualization. a) Analyze how Selection Sort and Bubble Sort would behave for this dataset. b) Compare their performance in terms of comparisons and swaps. c) Recommend the most suitable sorting method with proper justification. [50 Marks]

a) Behaviour of Selection Sort and Bubble Sort on this dataset

The sensor readings in this platform arrive continuously and in random order, since each sensor reports independently and there is no coordination that would keep the stream in any particular sequence. Before every trend-visualization cycle, this random batch of readings has to be sorted from scratch.

Selection Sort handles this by making $n-1$ passes over the batch of readings. On each pass, it scans the entire remaining unsorted portion to locate the smallest value, and then swaps that value into its correct position at the front of the unsorted region. Because it always scans the full remaining portion on every pass regardless of how the values are arranged, Selection Sort performs exactly the same number of comparisons no matter whether the readings happen to already be partly ordered or are completely scrambled — its comparison count depends only on how many readings there are, never on their arrangement. Crucially, it only swaps once per pass, so the total number of writes to the array stays small and fixed.

Bubble Sort instead makes repeated passes through the readings, comparing each pair of neighbouring values and swapping them whenever they are out of order, so that on every pass the largest remaining value gradually “bubbles” up towards the end. Since the incoming sensor readings are genuinely random rather than nearly sorted, Bubble Sort cannot take advantage of any existing order and ends up making close to the maximum possible number of comparisons on this dataset. What sets it apart from Selection Sort is that it swaps every single time it finds two neighbouring readings out of order, so the number of swaps it performs is tied directly to how scrambled the incoming batch is — and a fully random batch produces a great many out-of-order neighbouring pairs.

b) Comparing comparisons and swaps

For a batch of n sensor readings, both algorithms end up making essentially the same number of comparisons on this random dataset. Selection Sort always compares every element remaining in the unsorted region on each of its $n-1$ passes, which totals $n(n-1)/2$ comparisons regardless of the data. Bubble Sort, when the data is fully random rather than nearly sorted, also ends up making close to this same $n(n-1)/2$ comparisons in the worst case, since it has no shortcut to skip comparisons on scrambled data. So on this dataset, both algorithms sit at $O(n^2)$ time complexity purely from their comparisons, with little to separate them.

The clear difference is in swaps. Selection Sort swaps at most once per pass, giving it a fixed total of at most $n-1$ swaps across the whole sort, completely independent of how disordered the input is. Bubble Sort, by contrast, swaps every time it encounters an out-of-order neighbouring pair, and on a fully random sequence of readings this can add up to as many as $n(n-1)/2$ swaps — potentially tens or hundreds of times more writes than Selection Sort performs on the very same data. In short: comparisons are roughly tied between the two algorithms on this dataset, but Selection Sort keeps its swap count low and predictable while Bubble Sort's swap count is high and grows directly with how random the sensor readings are.

c) Recommended method and justification

Selection Sort is the more suitable choice for sorting this sensor-reading data. Since both algorithms carry the same $O(n^2)$ time complexity for comparisons on random data, neither has an edge purely on comparison count, so the decision comes down to swaps, and there Selection Sort has a clear and significant advantage: at most $n-1$ swaps versus up to $n(n-1)/2$ for Bubble Sort on this fully random data.

This matters in a data-analytics pipeline because every swap is effectively a write operation on the sensor-readings buffer, and this sort has to run periodically, every time a fresh batch of random readings needs to be prepared for trend visualization. Repeatedly incurring Bubble Sort's high, data-dependent swap cost on every such cycle adds unnecessary write overhead compared to Selection Sort's small, fixed cost. For this reason, between the two, Selection Sort is the better fit for this platform. It is worth noting, however, that both algorithms remain $O(n^2)$ overall, so if the number of sensor readings per batch grows very large, an $O(n \log n)$ algorithm such as Merge Sort or Heap Sort would ultimately scale better than either Selection Sort or Bubble Sort; but strictly between the two algorithms named in this question, Selection Sort is clearly preferable for this scenario.

Q37. A satellite tracking system evaluates coordinates of objects in space to find the closest pair using brute-force comparison. a) Explain how all coordinate pairs are evaluated. b) Derive the time complexity of the brute-force method. c) Discuss the limitations when the number of objects increases. [50 Marks]

a) How all coordinate pairs are evaluated

The brute-force closest-pair method works by examining every possible pair of tracked objects and computing the distance between them, keeping a running record of whichever pair has produced the smallest distance so far. For n objects, each with coordinates such as (x, y) or, in a full space-tracking context, (x, y, z) , the method uses two nested loops: the outer loop selects each object in turn, and the inner loop compares that object against every other object that comes later in the list. For each such pair, the straight-line (Euclidean) distance between the two sets of coordinates is calculated, and if this distance is smaller than the smallest one recorded so far, that pair becomes the new closest-pair candidate.

Restricting the inner loop to objects that come after the current one in the list ensures that every unique pair of objects is evaluated exactly once: no pair is checked twice, and no object is ever compared against itself.

Because every possible pairing is considered without exception, this exhaustive process is guaranteed to find the true closest pair among all the tracked objects, which is exactly the correctness guarantee that makes brute-force search reliable even though it is computationally expensive.

b) Deriving the time complexity of the brute-force method

With n tracked objects, the outer loop runs once for each object. For the first object, the inner loop compares it against the remaining $n-1$ objects; for the second, against the remaining $n-2$; and so on, down to the second-to-last object, which is compared against just one remaining object. Adding up this sequence of comparisons, $(n-1) + (n-2) + \dots + 1$, gives the well-known sum $n(n-1)/2$. Since each individual distance calculation between a pair of coordinates involves only a constant number of arithmetic operations — a fixed number of subtractions, multiplications, and one square root — the total running time is directly proportional to $n(n-1)/2$ pairwise checks. Dropping the constant factor and the lower-order term, as is standard in asymptotic analysis, gives an overall time complexity of $O(n^2)$. In terms of memory, the method needs very little beyond the list of coordinates itself: it only has to remember the smallest distance found so far and the identities of the two objects that produced it, so its auxiliary space complexity is $O(1)$.

c) Limitations when the number of objects increases

Because the brute-force method's running time grows with the square of the number of tracked objects, its performance degrades very quickly as a satellite-tracking catalog grows. Doubling the number of objects being tracked roughly quadruples the number of pairwise distance checks that must be performed, since the workload depends on n^2 rather than on n . For a handful of objects this overhead is trivial, but a real-world space-situational-awareness system may need to track many thousands of active satellites together with an even larger population of debris fragments, and it typically has to recompute closest-pair information frequently to flag potential collisions. At that scale, the number of pairs to evaluate becomes enormous — tracking 20,000 objects, for instance, would require checking on the order of 200 million pairs on every single update — making the brute-force approach far too slow to keep pace with the near-real-time demands of collision-risk monitoring.

This limitation is precisely what motivates moving to a more scalable divide-and-conquer closest-pair algorithm, which first sorts the objects by one coordinate and then recursively splits the space into halves, solving the closest-pair problem within each half and carefully checking only a narrow strip of candidates near the dividing line for a pair that crosses it. This reduces the overall time complexity to $O(n \log n)$, a substantial improvement over brute force's $O(n^2)$ that makes it practical to track and monitor collision risk for far larger populations of orbiting objects.